

# AI Resume Analyzer & Job Matcher

## Technical & Interview Guide

An explainable resume-to-job matching system built with FastAPI, React, and scikit-learn. This guide covers the architecture, the ML/NLP pipeline, the scoring methodology, a full code walkthrough, and a complete interview question bank.

Portfolio project by Ayush

# AI Resume Analyzer & Job Matcher — Interview Preparation Guide

This guide teaches the entire project from a beginner level up to an interview level. Everything here matches the **actual implementation** — no fabricated work. Where a part of the pipeline is optional (transformer embeddings, spaCy, the LLM), that is stated plainly so you never over-claim.

Golden rule for the interview: **the score is deterministic and explainable; the LLM is optional and only phrases things**. If you remember one sentence, remember that one.

## Part 1 — 30-second explanation

"It's a resume-to-job matcher. You upload a resume PDF and paste a job description, and it gives you an explainable match score — not a black-box number. It extracts and normalises skills from both sides, separates the job's required and preferred skills, matches them exactly, by synonym, and semantically, and combines six weighted signals into a score where every component is explained. It also lists your gaps, suggests projects to close them, and generates likely interview questions. I built it with FastAPI, React, and scikit-learn, with an optional LLM layer that only phrases the recommendations — it never decides the score."

## Part 2 — 2-minute explanation

"The problem I wanted to solve is that most 'AI resume scorers' just ask a language model to rate a resume out of 100, which is opaque and impossible to defend. So I built the scoring to be deterministic and explainable.

The pipeline: I extract text from the PDF with pdfplumber, clean it, and split it into sections like experience and projects. Then I extract skills using a curated taxonomy that normalises surface forms — so 'psql', 'Postgres', and 'PostgreSQL' all become one canonical skill with a category. I do the same on the job description, but I also split it into required versus preferred skills using cue phrases, because missing a required skill should hurt more.

Matching uses three signals: exact and normalised matches, which are the same check because both sides are already canonicalised, plus a semantic signal. For semantics I have a curated relatedness ontology — so PyTorch counts as related to a TensorFlow requirement — and cosine similarity over vectors, using TF-IDF by default and transformer embeddings when they're available.

The score is a weighted sum of six components — required coverage, overall overlap, preferred coverage, semantic similarity, project relevance, and experience relevance — each returned with its own sub-score and a one-line reason, so the UI can show exactly why. Skill signals carry about two-thirds and semantics one-third.

Finally there's an optional LLM layer — provider-agnostic between OpenAI and Anthropic — that generates recommendations and interview questions grounded in the real gaps, and it falls back to deterministic templates when no key is set. The frontend is React and TypeScript with a clean dashboard, and it's all containerised with Docker. I wrote 38 tests covering the pipeline and API."

## Part 3 — Full architecture

Three tiers:

1. **Frontend (React + TypeScript + Vite + Tailwind)**. A single page with a small state machine: input form → loading → dashboard. A typed API client mirrors the backend contract. Visualisations (score gauge, component bars) are pure SVG/CSS.
2. **Backend (FastAPI)**. Layered so each layer depends only on those below: - `api/` — thin HTTP layer (parse request, call service, shape response). - `schemas/` — Pydantic request/response models (the contract). - `services/` — orchestration: PDF

extraction, cleaning, resume/JD parsing, and the `analysis_service` that runs the whole pipeline. - `ml/` — the skill taxonomy, skill extractor, embeddings engine, and matching. - `scoring/` — documented weights + the explainable scoring engine. - `llm/` — provider-agnostic client + recommendations + interview + explanation. - `db/` — optional SQLAlchemy history. - `core/` — config, custom errors, logging. `utils/` — upload validation.

### 3. Optional PostgreSQL for analysis history (SQLite by default).

Key principle: the **LLM is a leaf**. The trunk (extract → skills → match → score) is deterministic and unit-tested; the LLM only decorates the result and has a non-LLM fallback.

## Part 4 — Complete data flow

```
Resume PDF
→ pdfplumber extracts the text layer           (services/pdf_extraction)
→ clean_text normalises whitespace/hyphenation (services/text_cleaning)
→ parse_resume splits sections + contact       (services/resume_parser)
→ extract_skills canonicalises resume skills    (ml/skill_extractor + taxonomy)

Job description
→ clean_text
→ parse_job_description splits required vs preferred (services/jd_parser)
→ extract_skills canonicalises JD skills

Resume skills + JD skills
→ match_skills:                               (ml/matching)
    exact/normalised (same canonical)
    + related (ontology siblings OR embedding cosine ≥ threshold)
    + missing (everything else)
→ semantic similarity of documents/sections    (ml/embeddings, cosine)
→ compute_score: 6 weighted, calibrated components (scoring/scoring_engine)
→ strengths / gaps                             (llm/explanation)
→ recommendations (LLM or template)            (llm/recommendations)
→ interview prep (LLM or template)             (llm/interview)

→ AnalysisResult (JSON) → optional history store (db/)
→ React dashboard renders the one JSON object
```

## Part 5 — ML / NLP concepts used

Only concepts actually in the project.

- **Tokenisation / text normalisation** — splitting and cleaning text so matching and vectorising see consistent input. (`text_cleaning.py`.)
- **Dictionary / phrase matching** — the skill extractor matches curated aliases with boundary rules; this is classic rule-based NLP, chosen for explainability.
- **Vector representations** — turning text into numbers so similarity is computable.
- **TF-IDF** — term frequency × inverse document frequency; a sparse, lexical vectorisation. Good when words overlap; blind to synonyms. (scikit-learn.)
- **Transformer embeddings** — `all-MiniLM-L6-v2` maps text to a dense vector that captures meaning; different words with similar meaning land close. (sentence-transformers — optional.)
- **Cosine similarity** — compares vector *direction* (angle), ignoring length; ideal for comparing a short resume section to a long JD.
- **Semantic similarity** — cosine over embeddings; catches relevant experience worded differently from the JD.
- **Calibration** — rescaling a backend's raw cosine range into an interpretable 0–1 score (heuristic bands, documented).
- **Named-entity recognition (NER)** — *optional*, via spaCy, only to pull the candidate's name; a regex heuristic runs if spaCy isn't installed.
- **LLM structured output** — prompting a model to return JSON constrained to real data; used only in the optional recommendation/interview layer.

**Not used (don't claim these):** we do **not** train a neural network, we do **not** fine-tune any model, and we do **not** use a learned classifier for scoring. Embeddings come from a **pretrained** model used as-is.

## Part 6 — Why each technology

- **Python** — one language for both the API and the ML/NLP work; best ecosystem for this (scikit-learn, pdfplumber, sentence-transformers).
- **FastAPI** — typed validation via Pydantic, async, and automatic OpenAPI docs.
- **pdfplumber** — reliable text-layer extraction without OCR overhead.
- **scikit-learn** — TF-IDF vectoriser and cosine similarity; the always-available semantic baseline.
- **sentence-transformers (optional)** — real transformer embeddings for semantic similarity; kept optional to keep the core light.
- **spaCy (optional)** — tokenisation + NER for name extraction; graceful fallback.
- **React + TypeScript** — component-based dashboard with a typed API contract.
- **Tailwind CSS** — fast, consistent styling without a heavy component library.
- **SQLAlchemy + PostgreSQL/SQLite** — optional history; SQLite for zero-setup dev, Postgres for production.
- **httpx** — provider-agnostic HTTP calls to the LLM (no vendor SDK lock-in).
- **Docker** — reproducible builds and simple deployment.
- **pytest** — fast, readable tests.

Nothing was added just to pad the stack — the ML libraries, spaCy, and the LLM are all optional and degrade gracefully.

---

## Part 7 — Important technical decisions

**Why not use only an LLM?** An LLM rating a resume is opaque, non-deterministic, and impossible to defend or unit-test. I made scoring a transparent weighted sum and demoted the LLM to phrasing recommendations, with a template fallback.

**Why use embeddings (and keep TF-IDF)?** TF-IDF only matches shared words, so it misses synonyms. Embeddings capture meaning. I keep TF-IDF as an always-available baseline and fallback, which also makes the TF-IDF-vs-embeddings comparison real.

**Why normalise skills?** Without it, "Postgres" and "PostgreSQL" wouldn't match and coverage would be wrong. A curated taxonomy makes normalisation exact and explainable.

**How is the score calculated?**  $100 \times \sum(\text{weight} \times \text{component})$  over six components; weights sum to 1 (validated at import); related skills get partial credit; semantic cosines are calibrated per backend.

**How do you handle missing skills?** They're separated into required vs preferred, weighted accordingly, surfaced explicitly in the UI, and drive the recommendations and gap-oriented interview questions.

**How do you handle synonyms?** The taxonomy's alias map plus the relatedness ontology (siblings like PyTorch/TensorFlow).

**How do you avoid hallucinated recommendations?** The LLM only ever sees the real matched/missing skills and is told to use only those; the deterministic template fallback is always available; and the LLM never affects the score.

**Why calibrate similarities?** Raw cosine between a resume and a JD is low even when relevant, and the useful range differs per backend, so each engine rescales its own observed band into 0-1.

---

## Part 8 — Interview questions & answers

Each question gives a **short** answer, a **detailed** answer, a **say-it-like-this** verbal version, and a **likely follow-up**. Answers match the real implementation.

### 8.1 Beginner

**Q1. What does your project do?** - *Short:* Scores how well a resume matches a job and explains why. - *Detail:* It extracts skills from a resume PDF and a job description, matches them, computes an explainable weighted score, and lists gaps, project ideas, and likely interview questions. - *Say it:* "You give it your resume and a job post; it tells you your match score, what you're missing, and what to do about it — and it shows the maths." - *Follow-up:* Why is the score better than asking ChatGPT? → It's deterministic and explainable, not a black box.

**Q2. What is the tech stack?** - *Short:* FastAPI + Python backend, React + TypeScript frontend, scikit-learn for similarity, Docker. - *Detail:* Python/FastAPI API, pdfplumber for PDFs, scikit-learn TF-IDF (optional sentence-transformers), optional SQLite/Postgres history, React/TS/Tailwind UI. - *Say it:* "Python and FastAPI on the back, React and TypeScript on the front, scikit-learn for the NLP similarity, all Dockerised." - *Follow-up:* Why FastAPI over Flask? → Typed validation, async, auto docs.

**Q3. How does a user use it?** - *Short:* Upload a PDF (or paste text) + paste a JD, click Analyze. - *Detail:* The frontend sends the

file and JD to `/api/analyze`; the backend runs the pipeline and returns one JSON result the dashboard renders. - *Say it*: "Three steps: add your resume, paste the job, hit analyze." - *Follow-up*: What if the PDF is a scan? → It's detected and rejected with a clear message.

**Q4. What is an API endpoint?** - *Short*: A URL the frontend calls to run an action. - *Detail*: e.g. `POST /api/analyze` accepts the resume + JD and returns the analysis JSON; `GET /api/health` reports status. - *Say it*: "Endpoints are the doors into the backend — analyze, health, skills, history." - *Follow-up*: GET vs POST? → GET reads, POST sends data / causes work.

**Q5. What is a PDF text layer?** - *Short*: The selectable text embedded in a PDF. - *Detail*: Exported PDFs contain real text I can read directly; scanned ones are just images with no text layer, which I detect and reject. - *Say it*: "Normal resumes have real text inside; scans are just pictures, so I handle that case explicitly." - *Follow-up*: How would you support scans? → OCR (e.g. Tesseract).

**Q6. What is JSON and why use it?** - *Short*: A text format for structured data exchanged between frontend and backend. - *Detail*: The API returns the whole analysis as one JSON object the React app renders; TypeScript types mirror its shape. - *Say it*: "The backend hands the frontend one JSON object with everything the dashboard needs." - *Follow-up*: How do you keep the types in sync? → A shared typed contract (Pydantic ↔ TypeScript interfaces).

**Q7. What is a skill taxonomy here?** - *Short*: A curated list mapping skill synonyms to one canonical name + category. - *Detail*: "psql/postgres/postgresql" → "PostgreSQL" (database). Enables correct matching and grouping. - *Say it*: "A dictionary that knows all the ways people write a skill and folds them into one." - *Follow-up*: What if a skill isn't in it? → It isn't matched — a known limitation.

**Q8. What does "matched / related / missing" mean?** - *Short*: Exact match, semantically related, or absent. - *Detail*: Matched = same canonical skill; related = a sibling skill or high similarity (partial credit); missing = neither. - *Say it*: "Green is a direct hit, amber is close, red is a gap." - *Follow-up*: Example of related? → Resume PyTorch vs JD TensorFlow.

**Q9. What is the match score out of?** - *Short*: 0 to 100. - *Detail*: A weighted sum of six components each in 0–1, times 100. - *Say it*: "It's a 0-to-100 number built from six explained parts." - *Follow-up*: What's the biggest part? → Required skill coverage (0.35).

**Q10. Why show a breakdown instead of just a number?** - *Short*: So the score is trustworthy and actionable. - *Detail*: Each component's sub-score and reason let a user see exactly what to improve and let me defend the number. - *Say it*: "A number alone is useless; the breakdown tells you what to fix." - *Follow-up*: Which component is easiest to game? → Keyword stuffing skills; that only moves coverage, and semantics/relevance temper it.

**Q11. What is Docker used for here?** - *Short*: Packaging the app so it runs the same everywhere. - *Detail*: A backend image, a frontend (nginx) image, and a compose file that also starts Postgres. - *Say it*: "One command spins up the whole stack in containers." - *Follow-up*: Multi-stage build? → Yes, Node builds the frontend, nginx serves it.

**Q12. What testing did you do?** - *Short*: 38 pytest tests across the pipeline and API. - *Detail*: Unit tests for cleaning, extraction, matching, scoring (with edge cases) and API tests via TestClient. - *Say it*: "I tested each stage and the endpoints, including the tricky edge cases." - *Follow-up*: A favourite test? → That 'ml' doesn't match inside 'html'.

## 8.2 Intermediate

**Q1. Walk me through the pipeline.** - *Short*: Extract → clean → parse → extract skills → match → score → recommend. - *Detail*: PDF→text (pdfplumber), clean, section-split, skill extraction with normalisation on both sides, JD required/preferred split, three-signal matching, six-component scoring, then LLM/template recommendations and interview prep. - *Say it*: (use the Part 4 data flow). - *Follow-up*: Where's the LLM? → Only at the end, optional, for phrasing.

**Q2. How does skill normalisation work?** - *Short*: An alias→canonical map applied to matched surface forms. - *Detail*: The taxonomy stores aliases; extraction matches longest-alias-first, records the surface form, and flags whether normalisation changed it. - *Say it*: "Every surface form folds into one canonical skill, and I keep the evidence." - *Follow-up*: How do you avoid 'ml' matching in 'html'? → Word-boundary look-arounds.

**Q3. How do you separate required vs preferred skills?** - *Short*: Cue phrases switch a current bucket while scanning lines. - *Detail*: "required/must have" → required region; "preferred/nice to have/bonus" → preferred; a skill in both counts as required. - *Say it*: "I read the JD top to bottom, flipping between required and preferred when I hit those cue words." - *Follow-up*: Inline cue like 'Required: Python'? → The line's skills are kept in that bucket.

**Q4. What's the difference between TF-IDF and embeddings?** - *Short*: TF-IDF is lexical (shared words); embeddings are semantic (meaning). - *Detail*: TF-IDF weights words by rarity and needs word overlap; MiniLM embeddings encode meaning so synonyms are close. - *Say it*: "TF-IDF sees words, embeddings see meaning — I use both." - *Follow-up*: When does TF-IDF fail? → 'PyTorch' vs 'deep learning frameworks'.

**Q5. What is cosine similarity and why it?** - *Short*: Cosine of the angle between vectors; length-invariant. - *Detail*:

$A \cdot B / (|A| |B|)$ ; ideal for comparing a short section to a long JD. - *Say it*: "It compares direction, not size, which suits resume vs JD." - *Follow-up*: Range? → 0–1 for our non-negative vectors.

**Q6. How does the scoring formula work?** - *Short*: Weighted sum of six components,  $\times 100$ . - *Detail*: required 0.35, overlap 0.20, preferred 0.10, semantic 0.15, project 0.10, experience 0.10; related = partial credit; weights validated to sum to 1. - *Say it*: "Skills carry two-thirds, semantics a third, and every part is explained." - *Follow-up*: Why those weights? → Design choice prioritising hard requirements; documented and tunable.

**Q7. How do you handle a missing LLM key?** - *Short*: Deterministic template fallback. - *Detail*: `LLMClient.is_enabled` is false → recommendations/interview come from a curated knowledge base; `meta.llm_used=false`. - *Say it*: "No key, no problem — it uses rule-based templates and says so." - *Follow-up*: If the LLM errors mid-request? → Caught, falls back, request still succeeds.

**Q8. How is the app made explainable?** - *Short*: Every component returns score + weight + reason; matches carry evidence. - *Detail*: The scoring engine emits per-component explanations; the UI renders the bars and reasons under the overall number. - *Say it*: "The dashboard is basically the audit trail of the score." - *Follow-up*: Could you export it? → Yes, it's just structured data.

**Q9. What validation do you do on uploads?** - *Short*: Extension, declared type, size, and the %PDF magic bytes. - *Detail*: All checks run before parsing; a file merely named .pdf is rejected if it lacks the signature. - *Say it*: "I don't trust the file — I check the real PDF signature and size first." - *Follow-up*: Why size first? → Avoid processing huge/abusive uploads.

**Q10. How do you keep frontend and backend in sync?** - *Short*: A typed contract: Pydantic models ↔ TypeScript interfaces. - *Detail*: `schemas/analysis.py` defines the response; `types.ts` mirrors it; the API client is typed. - *Say it*: "Both sides share one shape, so a mismatch shows up at compile time." - *Follow-up*: Auto-generate types? → Could from the OpenAPI schema; I kept it manual and small.

**Q11. Why is persistence optional?** - *Short*: Analysis is stateless; history is a bonus. - *Detail*: SQLite by default, Postgres in Docker; saving is best-effort and never breaks a request; only derived results are stored. - *Say it*: "History is nice-to-have, so it's off the critical path and can't break analysis." - *Follow-up*: Why not store the resume? → Privacy — only the result is kept.

**Q12. What are the main failure modes and how are they handled?** - *Short*: Bad/scanned PDF, empty input, oversized file, LLM failure — all clean errors or fallbacks. - *Detail*: Custom `AppError` types map to HTTP codes via a global handler; the LLM path always falls back. - *Say it*: "Every expected failure returns a clean JSON message, never a stack trace." - *Follow-up*: Unexpected bug? → Caught by a last-resort handler returning a 500 JSON.

### 8.3 Advanced

**Q1. Why is a deterministic score better than an LLM score?** - *Short*: Reproducible, testable, explainable, cheap. - *Detail*: Same inputs → same output; I can unit-test it, decompose it, and defend it. An LLM number varies run to run and can't be audited. - *Say it*: "I can put the score on trial — an LLM's number I can't." - *Follow-up*: Any downside? → Less nuanced than a strong LLM's holistic read; I mitigate with semantic components.

**Q2. How do you prevent short ambiguous tokens (Go, C, R) from false-matching?** - *Short*: Require a list-context delimiter for those aliases. - *Detail*: Boundary look-arounds handle substrings; ambiguous English-word aliases additionally must sit next to a comma/slash/pipe/bracket/newline. - *Say it*: "'Go' only counts when it looks like a list item, not in prose like 'go to production'." - *Follow-up*: Trade-off? → A skills line starting mid-prose could be missed — rare and acceptable.

**Q3. How does longest-alias-first matching avoid double counting?** - *Short*: Matched spans are consumed so shorter aliases can't re-match them. - *Detail*: Aliases are sorted longest-first; a match blanks its span, so "spring boot" is consumed before "spring", and "node.js" before "node". - *Say it*: "I match the most specific alias first and cross it out." - *Follow-up*: Complexity? → Linear in aliases  $\times$  text; pre-compiled patterns.

**Q4. Why calibrate cosine similarities, and how?** - *Short*: Raw cosine is low/backend-dependent; rescale to 0–1. - *Detail*: Each engine rescales its observed useful band (TF-IDF  $\sim [0, 0.35]$ , MiniLM  $\sim [0.2, 0.7]$ ) and clamps, so the component is interpretable. - *Say it*: "A raw 0.3 cosine can mean 'very relevant' — I map each backend's real range onto 0-to-1." - *Follow-up*: Isn't that arbitrary? → It's a documented heuristic, not learned; I'd fit it from labelled data given a dataset.

**Q5. How do you combine the ontology and embedding relatedness?** - *Short*: Related if either fires; ontology gives the reason. - *Detail*: Unmatched JD skill → check sibling group  $n$  resume skills (ontology, reason "same family"), else embedding cosine  $\geq$  threshold (semantic). - *Say it*: "Ontology for reliable, explainable siblings; embeddings for the open-ended rest." - *Follow-up*: Why not embeddings only? → They can't run offline here and give no human-readable reason.

**Q6. How would this scale to thousands of resumes per JD?** - *Short*: Precompute JD skills/embeddings; batch-encode; cache. - *Detail*: Extract JD once, reuse; batch resume embeddings; the taxonomy match is cheap; add a queue and horizontal API replicas. - *Say it*: "Compute the JD side once and batch the resume side; the heavy bit is embeddings, which batch well." - *Follow-up*: Bottleneck? → Embedding inference — cache and batch, or use a vector DB for reuse.



**Q7. What are the security considerations?** - *Short:* Validate uploads, keep secrets in env, restrict CORS, non-root, don't store resumes. - *Detail:* Magic-byte + size checks pre-parse; `.env` git-ignored; CORS to configured origins; container runs as non-root; only derived results persisted. - *Say it:* "I don't trust the upload, I don't leak secrets, and I don't keep the resume." - *Follow-up:* Next step? → Auth + rate limiting + AV scan on uploads.

**Q8. Where is the biggest source of error, and how would you reduce it?** - *Short:* Skill recognition (taxonomy coverage) and section detection. - *Detail:* Out-of-taxonomy skills are invisible; unusual layouts confuse headings. A learned tagger + more robust layout parsing would help. - *Say it:* "My weakest link is anything outside the taxonomy — I'd add a learned skill tagger to catch the long tail." - *Follow-up:* How to measure it? → Label a set of resume/JD pairs and track precision/recall of extraction.

**Q9. How is the LLM layer provider-agnostic?** - *Short:* One client with per-provider HTTP calls behind a common method. - *Detail:* `generate / generate_json` dispatch to OpenAI or Anthropic via httpx; swapping providers is a config change; JSON parsing tolerates fences/prose. - *Say it:* "The rest of the app just calls `generate_json`; the provider is a setting." - *Follow-up:* Add a local model? → Implement one more branch with the same interface.

**Q10. How do you keep the LLM from hallucinating skills?** - *Short:* Constrain it to the real skill lists; template fallback; score independent. - *Detail:* The prompt passes exact matched/missing skills and says "use only these"; JSON is validated; the score never depends on the LLM. - *Say it:* "It only ever sees the real gaps and is told to stay inside them." - *Follow-up:* If it still drifts? → Validation rejects the shape and I fall back to templates.

**Q11. Why layer the backend the way you did?** - *Short:* Separation of concerns and testability. - *Detail:* API depends on services, services on ml/scoring/llm, none of which know about HTTP — so the whole pipeline is unit-testable without a server. - *Say it:* "HTTP is a thin shell around a pure pipeline I can test directly." - *Follow-up:* Downside? → More files; worth it for clarity.

**Q12. How would you A/B test different weightings?** - *Short:* Make weights a versioned config and log outcomes. - *Detail:* Weights already live in one module; expose a version, record which produced which score, and compare against a labelled or feedback signal. - *Say it:* "The weights are one small config, so versioning and comparing them is straightforward." - *Follow-up:* What's the label? → Recruiter/user feedback or interview-callback outcomes.

## 8.4 Project-specific

**Q1. Why did you pick this project?** - *Short:* It combines real NLP with a clear product and an explainability angle. - *Detail:* It let me build a genuine pipeline (not a CRUD app), practise NLP and scoring design, and produce something I can defend end to end. - *Say it:* "I wanted one project with real ML depth that I fully understand." - *Follow-up:* Hardest part? → Making the score honest and explainable.

**Q2. What was the hardest bug?** - *Short:* Skill-boundary false positives/negatives. - *Detail:* 'TF-IDF' matched 'TensorFlow' via a 'tf' alias; trailing periods blocked matches. I fixed the boundary rules and removed the bad alias — the tests caught both. - *Say it:* "Getting skill matching precise — like 'tf-idf' wrongly hitting TensorFlow — took careful boundary rules." - *Follow-up:* How did you catch it? → A failing unit test on real sample text.

**Q3. What would you do differently?** - *Short:* Add a learned skill tagger and a labelled evaluation set. - *Detail:* The taxonomy is precise but limited; a small dataset would let me learn weights/calibration and measure extraction quality. - *Say it:* "I'd invest in evaluation data so I could learn, not guess, the weights." - *Follow-up:* Why not now? → No labelled data; I chose an honest heuristic.

**Q4. Which part are you most proud of?** - *Short:* The explainable scoring engine. - *Detail:* Six components each carrying score, weight, and reason — turning a number into an audit trail. - *Say it:* "That the score explains itself is the whole point, and it works." - *Follow-up:* Most reused code? → The semantic engine interface.

**Q5. How did you keep it 'at your level' but still impressive?** - *Short:* Deterministic, explainable design over flashy but opaque ML. - *Detail:* I chose components I fully understand (taxonomy, TF-IDF, cosine, weighted sum) and made the heavy ML optional. - *Say it:* "Ten well-engineered pieces I can explain beat a hundred I can't." - *Follow-up:* Example of restraint? → LLM optional, not central.

**Q6. How is the sample data made and why synthetic?** - *Short:* Hand-written fake resumes/JDs to avoid using real people's data. - *Detail:* `data/` holds synthetic text; a script renders PDFs for demos/tests. - *Say it:* "All demo data is invented — no real resumes." - *Follow-up:* Privacy angle? → Also why I don't store uploaded resumes.

**Q7. How do you show the score is trustworthy in the UI?** - *Short:* The "Why this score" panel with bars and reasons. - *Detail:* Each component's percentage, weight, and one-line explanation sit right under the overall gauge. - *Say it:* "The breakdown is literally next to the number." - *Follow-up:* Colour meaning? → Traffic-light by component strength.

**Q8. What does the meta/transparency footer show?** - *Short:* Which semantic backend ran and whether the LLM was used. - *Detail:* `embedding_backend` (tfidf/embeddings) and `llm_used`, plus any warnings (e.g. missing projects section). - *Say it:* "It's honest about how the result was produced." - *Follow-up:* Why surface the backend? → So a low semantic score under TF-IDF is

understood, not mistaken for a bad candidate.

**Q9. How would you demo this in 60 seconds?** - *Short:* Click "Load a sample", Analyze, walk the dashboard top-down. - *Detail:* Show the gauge, the breakdown reasons, matched/missing chips, then the recommendations and interview questions. - *Say it:* "Sample, analyze, and read the story the dashboard tells." - *Follow-up:* Without a backend? → The sample flow still needs the API; I can run it locally.

**Q10. What did you learn building it?** - *Short:* NLP matching precision, scoring design, and clean architecture. - *Detail:* Boundary rules matter a lot; explainability is a design constraint; optional dependencies keep a project runnable. - *Say it:* "I learned how much care 'simple' skill matching actually takes." - *Follow-up:* Next skill to add? → A learned evaluation loop.

**Q11. How long did it take and how did you structure the work?** - *Short:* Built in phases: architecture → backend pipeline → scoring → LLM → UI → tests → docs. - *Detail:* Each phase was tested before the next, so bugs surfaced early. - *Say it:* "I built it like a real project — incremental, tested per stage." - *Follow-up:* CI? → Tests are ready to wire into CI.

**Q12. Could this be a real product?** - *Short:* The core could; it needs auth, scale, and a bigger skill model. - *Detail:* The pipeline and UX are product-grade; production needs accounts, rate limiting, evaluation, and taxonomy expansion. - *Say it:* "It's a strong MVP — the honest scoring is a real differentiator." - *Follow-up:* Monetisation? → Per-seat for candidates or ATS integration.

## 8.5 ML / NLP

**Q1. Explain TF-IDF.** - *Short:* Weighs a word by its frequency here × rarity across documents. - *Detail:* Common-everywhere words get low weight, distinctive words high; documents become sparse weighted vectors compared by cosine. - *Say it:* "It highlights the words that make a document distinctive." - *Follow-up:* Weakness? → No notion of meaning/synonyms.

**Q2. What is a word/sentence embedding?** - *Short:* A dense vector capturing meaning. - *Detail:* A transformer (MiniLM) maps text to ~384 numbers so semantically similar text is nearby in the space. - *Say it:* "Meaning as coordinates — similar meanings sit close together." - *Follow-up:* Dimensionality here? → MiniLM is 384-d.

**Q3. Why transformers for embeddings?** - *Short:* Context-aware representations beat bag-of-words. - *Detail:* Self-attention lets the model weigh context, so the same word in different contexts embeds differently; sentence-transformers pool this into one vector. - *Say it:* "Transformers read words in context, which is why the vectors carry meaning." - *Follow-up:* Did you train it? → No — pretrained, used as-is.

**Q4. What is cosine similarity mathematically?** - *Short:* Dot product over the product of magnitudes. - *Detail:*  $\cos \theta = \frac{A \cdot B}{(|A| |B|)}$ ; on L2-normalised vectors it's just the dot product, which is why I normalise embeddings. - *Say it:* "Angle between vectors; on unit vectors it's a dot product." - *Follow-up:* Why not Euclidean? → It's magnitude-sensitive; length shouldn't matter here.

**Q5. Is your skill extraction NLP or just string matching?** - *Short:* Rule-based NLP — dictionary/phrase matching with boundary logic. - *Detail:* It's deliberately not a learned model, for explainability; it's still NLP (tokenisation, normalisation, phrase matching). - *Say it:* "It's classic rule-based NLP, chosen because it's explainable and needs no training data." - *Follow-up:* When would you switch to ML? → With labelled data to catch out-of-taxonomy skills.

**Q6. Do you use NER? Where?** - *Short:* Optionally, via spaCy, for the candidate's name only. - *Detail:* If spaCy is installed, a PERSON entity near the top is used; otherwise a regex heuristic. It never affects scoring. - *Say it:* "NER is a small optional nicety for the name, with a regex fallback." - *Follow-up:* Why not NER for skills? → Precision/explainability of the taxonomy.

**Q7. How do you evaluate quality without labels?** - *Short:* Unit tests on known cases + ordering checks. - *Detail:* I assert specific extractions/matches and that a better-fitting resume always outscores a worse one; true precision/recall needs a labelled set. - *Say it:* "I test known-good cases and monotonic ordering; formal metrics need labels I didn't have." - *Follow-up:* What dataset would you build? → Resume/JD pairs with recruiter judgements.

**Q8. What's the difference between semantic and exact matching here?** - *Short:* Exact = same canonical skill; semantic = similar meaning. - *Detail:* Exact/normalised is a set check on canonical names; semantic is cosine over vectors, used for related skills and the relevance components. - *Say it:* "One is 'the same skill', the other is 'about the same thing'." - *Follow-up:* Which do you trust more? → Exact; semantic only earns partial credit.

**Q9. Why not just embed the whole resume and JD and call that the score?** - *Short:* Too coarse and not explainable. - *Detail:* One cosine number hides which skills matter; my component score ties back to specific skills and sections. - *Say it:* "A single similarity is a vibe, not an explanation." - *Follow-up:* Do you use whole-doc similarity at all? → Yes, as one 0.15-weighted component.

**Q10. What preprocessing matters most for embeddings vs TF-IDF?** - *Short:* TF-IDF needs clean tokens; embeddings want natural text. - *Detail:* TF-IDF benefits from stopword handling and consistent tokens; embeddings prefer natural casing/punctuation, so I keep display text separate from the match-normalised text. - *Say it:* "I keep two text forms — aggressive



for matching, natural for embeddings." - *Follow-up*: Lemmatisation? → Not needed and would harm skill tokens.

**Q11. How big is the embedding model and why that one?** - *Short*: MiniLM (all-MiniLM-L6-v2), ~80–90 MB, fast and good enough. - *Detail*: It's a strong speed/quality trade-off for short-text similarity on CPU; bigger models add latency for marginal gains here. - *Say it*: "MiniLM is the sweet spot — small, fast, and accurate for this." - *Follow-up*: Upgrade path? → A larger sentence-transformer if quality demanded.

**Q12. What is the vector for a skill vs a document?** - *Short*: Same mechanism, different input length. - *Detail*: I embed/encode both short skill strings (for relatedness) and longer section/document text (for relevance); cosine compares them. - *Say it*: "Skills and documents both become vectors; only the length differs." - *Follow-up*: Do single-word TF-IDF vectors work? → Poorly — that's why ontology handles skill relatedness in the fallback.

## 8.6 Backend / API

**Q1. Why FastAPI?** - *Short*: Typed validation, async, auto OpenAPI docs. - *Detail*: Pydantic models validate requests/responses; Swagger UI comes free; async suits I/O like LLM calls. - *Say it*: "It gives me typed contracts and docs for free in the same language as my ML." - *Follow-up*: Async everywhere? → The analyze route reads the upload async; the CPU pipeline is sync.

**Q2. How do you validate requests?** - *Short*: Pydantic models + explicit upload checks. - *Detail*: `AnalyzeTextRequest` enforces min lengths; the PDF route runs magic-byte/size/type checks before parsing. - *Say it*: "Pydantic guards the JSON, and I hand-check the file bytes." - *Follow-up*: Where do errors become responses? → Global exception handlers.

**Q3. How is error handling structured?** - *Short*: Custom `AppError` subclasses mapped to HTTP codes by a global handler. - *Detail*: Each error carries a status and code; a catch-all handler returns a JSON 500 for anything unexpected — no stack traces leak. - *Say it*: "Expected errors are typed and become clean JSON; nothing raw reaches the user." - *Follow-up*: Example code? → `invalid_file` → 422.

**Q4. How does CORS work here?** - *Short*: Allowed origins come from config. - *Detail*: `CORSMiddleware` uses `CORS_ORIGINS`; in Docker the frontend nginx proxies `/api` so it's same-origin. - *Say it*: "CORS is locked to configured origins, or avoided via a proxy in prod." - *Follow-up*: Symptom of a CORS bug? → Browser blocks the request; fix the origin list.

**Q5. Why keep the route layer thin?** - *Short*: Testability and separation. - *Detail*: Routes parse and delegate to `analysis_service`, which is pure Python I can test without HTTP. - *Say it*: "Business logic lives in services, not routes." - *Follow-up*: How do you test routes then? → `TestClient` for happy/error paths.

**Q6. How is configuration managed?** - *Short*: pydantic-settings from `env/.env`, cached. - *Detail*: One `settings` object; no module reads `os.environ` directly; every setting has a default. - *Say it*: "All config funnels through one typed settings object." - *Follow-up*: Secrets? → Env vars, never committed.

**Q7. How would you add authentication?** - *Short*: JWT or session middleware on the API. - *Detail*: Add an auth dependency to protected routes, issue tokens, and scope history per user. - *Say it*: "A FastAPI dependency guarding routes plus per-user history." - *Follow-up*: Where store users? → The existing DB.

**Q8. How does the optional DB integrate without being required?** - *Short*: Behind a flag; best-effort saves. - *Detail*: `ENABLE_HISTORY` gates it; init runs in the lifespan handler and won't block startup; save/list swallow errors. - *Say it*: "History is opt-in and can fail without touching analysis." - *Follow-up*: SQLite→Postgres switch? → Just change `DATABASE_URL`.

**Q9. What does `/api/analyze` return?** - *Short*: One JSON object with score, breakdown, skills, recs, interview, meta. - *Detail*: Matches the `AnalysisResponse` schema; the frontend renders entirely from it. - *Say it*: "Everything the dashboard needs in a single response." - *Follow-up*: Why one call not many? → Simpler client, atomic result.

**Q10. How do you handle large or slow requests?** - *Short*: Size limits + time-boxed LLM + async upload read. - *Detail*: Uploads capped at `MAX_UPLOAD_MB`; LLM has a timeout and fallback so a slow provider can't hang the request. - *Say it*: "I cap uploads and time-box the only external call." - *Follow-up*: Long term? → Background workers for heavy batches.

**Q11. Why httpx instead of a vendor SDK for the LLM?** - *Short*: No lock-in; provider is a config branch. - *Detail*: Direct HTTP keeps dependencies minimal and swapping OpenAI/Anthropic trivial. - *Say it*: "One thin client, two providers, easy to extend." - *Follow-up*: Retries? → Time-boxed with fallback; add backoff if needed.

**Q12. How is the API documented?** - *Short*: Automatic OpenAPI/Swagger at `/docs`. - *Detail*: FastAPI generates it from the Pydantic models and route signatures. - *Say it*: "Swagger UI is generated from the typed models." - *Follow-up*: Client generation? → Types can be generated from the OpenAPI schema.

## 8.7 System design

**Q1. Design this to serve 1M analyses/day.** - *Short*: Stateless API replicas + queue + caching + vector store. - *Detail*:

Horizontally scale the stateless API behind a load balancer; offload heavy analysis to workers via a queue; cache JD parses and embeddings; store vectors in a vector DB; managed Postgres for history. - *Say it*: "The API is stateless, so I scale out and push the heavy embedding work to workers with caching." - *Follow-up*: Hot path? → Embedding inference — batch and cache.

**Q2. Where would you cache, and what?** - *Short*: JD parses, embeddings, and repeat analyses. - *Detail*: Key by content hash; cache JD skill extraction and section/skill embeddings; optionally memoise identical resume+JD pairs. - *Say it*: "Cache anything derived from the same text — JD parses and vectors especially." - *Follow-up*: Invalidation? → Content-hash keys, so it's naturally immutable.

**Q3. How do you make embeddings fast at scale?** - *Short*: Batch encoding, GPU/warm workers, precompute JD side. - *Detail*: Encode many texts per call; keep model warm in dedicated workers; compute the JD once and reuse across candidates. - *Say it*: "Batch the encodes and only embed the JD once per posting." - *Follow-up*: Model too slow? → Distil/quantise or a smaller model.

**Q4. How would you evaluate and improve the score in production?** - *Short*: Collect feedback, log components, learn weights. - *Detail*: Capture recruiter/user feedback or callback outcomes, log per-component scores, then fit weights/calibration and monitor drift. - *Say it*: "Instrument the components, gather outcome labels, then learn the weights." - *Follow-up*: Guard against gaming? → Weight semantics/relevance, not just keywords.

**Q5. How do you keep personal data safe?** - *Short*: Don't store resumes; encrypt at rest/in transit; access controls. - *Detail*: Only derived results are stored; add TLS, encryption, retention limits, and per-user access if accounts exist. - *Say it*: "I persist the analysis, not the resume, and I'd add encryption and retention rules." - *Follow-up*: GDPR? → Right-to-delete is easy since little is stored.

**Q6. How would you support many JD formats and languages?** - *Short*: Pluggable parsers; multilingual embeddings. - *Detail*: Abstract JD parsing behind an interface; swap in a multilingual sentence-transformer and localise the taxonomy. - *Say it*: "Parsing is already isolated, and the embedding model can go multilingual." - *Follow-up*: Taxonomy per language? → Alias sets per locale.

**Q7. What's your rollout/versioning strategy for scoring changes?** - *Short*: Version the weights/calibration and shadow-test. - *Detail*: Run new weights in shadow, compare distributions, then ramp; store the version with each result. - *Say it*: "Version the config, shadow it, compare, then roll out." - *Follow-up*: Reproduce an old score? → The stored version + inputs.

**Q8. Where are the single points of failure?** - *Short*: The LLM provider and (if used) the DB. - *Detail*: Both are non-critical: LLM has a template fallback; DB is best-effort. The API itself scales horizontally. - *Say it*: "The only externals are optional and degrade gracefully." - *Follow-up*: Provider outage? → Automatic template fallback.

**Q9. How do you monitor it?** - *Short*: Health checks, latency/error metrics, component distributions. - *Detail*: `/api/health`, request metrics, LLM fallback rate, and score-component histograms to catch drift. - *Say it*: "I'd watch latency, the LLM fallback rate, and score distributions." - *Follow-up*: Alert on? → Spike in fallbacks or extraction errors.

**Q10. Batch mode: rank 500 resumes for one JD?** - *Short*: Parse JD once; batch-embed resumes; sort by score. - *Detail*: Reuse JD extraction/embeddings; batch the resume side; return a ranked list — a natural extension of the current pipeline. - *Say it*: "It's the same pipeline with the JD computed once and resumes batched." - *Follow-up*: Fairness? → Audit for biased signals; keep it skills-based.

## 8.8 Follow-up / trick questions

**Q1. "Isn't this just keyword matching?"** - *Short*: No — it's normalised matching plus semantics and weighted scoring. - *Detail*: Normalisation, required/preferred weighting, ontology relatedness, and embedding similarity go well beyond keyword counting. - *Say it*: "Keywords are step one; normalisation, semantics, and weighting are the real work." - *Follow-up*: Prove semantics matter → PyTorch resume vs TensorFlow JD → related.

**Q2. "Why not just let GPT do the whole thing?"** - *Short*: Opaque, non-deterministic, undefendable. - *Detail*: I'd lose reproducibility, testability, and explainability — the entire value proposition. - *Say it*: "Then I couldn't explain or test the score, which is the point." - *Follow-up*: Any LLM at all? → Yes, optional, for phrasing only.

**Q3. "Your semantic score was 4% — is the model broken?"** - *Short*: No — that's the TF-IDF fallback on lexically different text. - *Detail*: TF-IDF sees no shared words between specific projects and generic JD language; embeddings raise it. The footer shows which backend ran. - *Say it*: "That's TF-IDF being honest about zero word overlap; embeddings fix it." - *Follow-up*: So why keep TF-IDF? → Always-available baseline and fallback.

**Q4. "What if two skills have the same abbreviation?"** - *Short*: The taxonomy picks one canonical; ambiguity is documented. - *Detail*: I avoid the worst collisions (dropped 'cv' for Computer Vision vs curriculum vitae); genuinely ambiguous cases are a known limitation. - *Say it*: "I curate away the dangerous abbreviations and document the rest." - *Follow-up*: 'CV' example → intentionally not an alias.

**Q5. "Can I trick it by pasting the JD into my resume?"** - *Short*: Coverage would rise, but it's honest about that. - *Detail*:

Keyword stuffing lifts coverage; semantic/project/experience relevance and the visible breakdown temper and expose it. - *Say it*: "You can inflate coverage, but the relevance components and the visible reasons push back." - *Follow-up*: Harden it? → Penalise skill lists with no supporting context.

**Q6. "Why six components and not five or ten?"** - *Short*: Enough to be meaningful, few enough to explain. - *Detail*: They cover required/preferred/overall skills plus semantic/project/ experience relevance — the axes that matter, each explainable. - *Say it*: "Each component answers a distinct question a recruiter would ask." - *Follow-up*: Redundancy? → Project vs experience overlap is intentional and low-weighted.

**Q7. "Your weights look arbitrary."** - *Short*: They're a documented design choice, validated to sum to 1. - *Detail*: Skills two-thirds, semantics one-third, required dominant; I'd learn them from labelled data if I had it. - *Say it*: "They're principled defaults, not magic — and easy to tune or learn." - *Follow-up*: Prove they sum to 1? → An import-time assertion.

**Q8. "What breaks if the LLM returns garbage?"** - *Short*: Nothing — it's validated and falls back. - *Detail*: Non-JSON or missing keys raise `LLMError`; the template path runs and `llm_used=false`. - *Say it*: "Bad output is caught and I quietly fall back to templates." - *Follow-up*: Silent to the user? → The footer flags rule-based mode.

**Q9. "Does semantic similarity prove the candidate has the skill?"** - *Short*: No — that's why it's 'related', with partial credit only. - *Detail*: Similarity suggests relevance, not possession; exact matches carry full weight, related only half. - *Say it*: "Related is a hint, not proof, so it earns half credit." - *Follow-up*: Could it mislead? → Capped weight limits the damage.

**Q10. "How do you know your extraction is correct?"** - *Short*: Tests on known text + evidence recorded per match. - *Detail*: Unit tests assert specific extractions and boundary behaviour; each match stores its surface form for auditing. - *Say it*: "I test it on known inputs and keep the evidence for every match." - *Follow-up*: Formal metric? → Needs a labelled set; a documented next step.

**Q11. "Why not store resumes to improve the model?"** - *Short*: Privacy; and there's no trained model to improve. - *Detail*: The system is rule/vector based with a pretrained embedder — no training loop — so I keep only derived results. - *Say it*: "There's nothing to train, and not storing resumes is the safer default." - *Follow-up*: If you added learning? → Explicit consent + anonymisation.

**Q12. "What's the weakest claim you could defend here?"** - *Short*: That semantic relevance is robust under the TF-IDF fallback. - *Detail*: It's compressed without embeddings; I'm upfront that those components shine with the transformer backend, and the ordering still holds. - *Say it*: "Under TF-IDF the semantic parts are weak-but-honest; I don't oversell them." - *Follow-up*: Fix? → Ship with embeddings enabled in production.

---

## Part 9 — Code walkthrough (revise before an interview)

---

For each key file: what it does, its main function(s), input/output, the important logic, why it exists, and a likely question.

`services/pdf_extraction.py` — `extract_text_from_pdf(data: bytes) -> str` - In: PDF bytes. Out: text. Logic: pdfplumber per-page text; near-empty → raise `PDFExtractionError`. Why: robust text-in. Q: "How do you detect a scanned PDF?"

`services/text_cleaning.py` — `clean_text / normalise_for_matching` - In/Out: str→str. Logic: whitespace/hyphen/bullet normalisation; a separate match-form that preserves `c++`, `node.js`. Why: consistent downstream input. Q: "Why two text forms?"

`ml/skills_taxonomy.py` — `taxonomy + RELATED_MAP` - Data: canonical skills, aliases, categories, sibling groups. Logic: build alias→skill and relatedness maps at import. Why: normalisation + explainable relatedness. Q: "How does normalisation stay explainable?"

`ml/skill_extractor.py` — `extract_skills(text) -> list[ExtractedSkill]` - In: text. Out: canonical skills with evidence. Logic: longest-alias-first, boundary look-arounds, ambiguous list-context guard, span consumption. Why: the core extraction. Q: "How do you avoid 'ml' in 'html'?"

`services/jd_parser.py` — `parse_job_description(text) -> StructuredJD` - In: JD text. Out: required/preferred skills + years/education. Logic: cue-phrase region split, required-wins dedupe. Why: importance weighting. Q: "How do you split required vs preferred?"

`ml/embeddings.py` — `get_semantic_engine()`, `similarity`, `similarity_matrix`, `calibrate` - In: strings. Out: cosine similarities / calibrated scores. Logic: TF-IDF or transformer behind one interface; auto-select + per-backend calibration. Why: semantic signal. Q: "TF-IDF vs embeddings?"

`ml/matching.py` — `match_skills(resume_skills, jd, engine) -> MatchReport` - In: skills + JD + engine. Out: matched/related/missing + coverage counts. Logic: exact set match; related via ontology OR cosine≥threshold; rest missing. Why: the matching engine. Q: "How is 'related' decided?"

`scoring/scoring_engine.py` — `compute_score(...)` -> `ScoreResult` - In: match report + texts + engine. Out: overall + six explained components. Logic: coverage ratios (+partial related credit), calibrated semantics, weighted sum. Why: explainable score. Q: "Walk me through the formula."

`scoring/weights.py` — `WEIGHTS`, `validate_weights()` - Data: the six weights + labels. Logic: import-time sum-to-1 assertion. Why: single source of truth. Q: "Why these weights?"

`llm/client.py` — `LLMClient.generate` / `generate_json` - In: prompts. Out: text/JSON. Logic: provider dispatch (OpenAI/Anthropic), time-box, JSON extraction, `LLMError` on failure. Why: optional, swappable LLM. Q: "How is it provider-agnostic and safe?"

`llm/recommendations.py` / `llm/interview.py` - In: match report (+role). Out: recs / interview prep. Logic: LLM if enabled else templates grounded in real gaps. Why: actionable, hallucination-safe output. Q: "How do you avoid hallucinated advice?"

`services/analysis_service.py` — `analyze(resume_text, jd_text)` -> `dict` - In: two texts. Out: the full `AnalysisResult`. Logic: orchestrates the whole pipeline; empty-input guards. Why: one testable entry point. Q: "Where does the pipeline live?"

`app/main.py` + `app/api/routes.py` - Logic: FastAPI app, CORS, lifespan DB init, global error handlers; thin routes delegating to the service. Why: clean HTTP boundary. Q: "Why keep routes thin?"

---

## Part 10 — Final mental checklist before an interview

---

- I can give the 30-second and 2-minute pitches.
- I can draw the data flow from memory.
- I can explain TF-IDF vs embeddings and cosine similarity.
- I can justify the six components and the weights.
- I can say exactly what the LLM does — and doesn't — do.
- I can name the limitations honestly (taxonomy coverage, TF-IDF fallback, heuristic calibration).
- I never claim I trained a model; embeddings are pretrained and optional.

# Appendix A — System Architecture

---

# System Architecture

This document explains how the AI Resume Analyzer & Job Matcher is put together, why each piece exists, and the trade-offs behind the main decisions. It is written to be read start-to-finish before an interview.

## 1. The problem

A candidate has a resume (a PDF) and a job description (free text). They want an honest, explainable answer to one question: *how well do I fit this role, and what should I do about the gaps?* A single number ("74%") is useless if it can't be defended. So the whole system is built around **explainability**: every score decomposes into named components, and every component traces back to concrete evidence (a matched skill, a similarity value, a covered requirement).

## 2. High-level shape



The key design principle: **the LLM is a leaf, not the trunk**. The trunk (extraction → skills → matching → score) is deterministic and testable. The LLM only decorates the result with natural-language recommendations and interview questions, and even that has a non-LLM fallback. This is a deliberate choice — it is what makes the project defensible rather than "I asked GPT to rate a resume."

## 3. Backend layout



```

backend/app/
  main.py      FastAPI app: CORS, routers, global exception handlers
  api/         HTTP layer – request parsing, response shaping only
  core/        config (env-driven settings), custom errors, logging
  schemas/     Pydantic request/response models (the API contract)
  services/    orchestration: pdf_extraction, text_cleaning,
               resume_parser, jd_parser, analysis_service
  ml/          skills_taxonomy, skill_extractor, embeddings, similarity
  scoring/     weights (documented) + scoring_engine (the formula)
  llm/         provider-agnostic client + recommendations + interview
  db/          optional SQLAlchemy models + session (analysis history)
  utils/       file validation helpers

```

Each layer has one job and depends only on the layers "below" it. The `api` layer never contains business logic; the `services` / `ml` / `scoring` layers never know about HTTP. This separation is what lets the entire scoring pipeline be unit-tested without starting a web server (see `tests/`).

## 4. The pipeline, stage by stage

- PDF extraction** (`services/pdf_extraction.py`) — `pdfplumber` reads text page by page. If a PDF is scanned/image-only there is no text layer, so we detect the empty result and raise a clean, user-facing error instead of crashing.
- Text cleaning** (`services/text_cleaning.py`) — normalise whitespace, fix hyphenation across line breaks, strip control characters. Cleaning happens before any NLP so downstream stages see consistent input.
- Section detection** (`services/resume_parser.py`) — a deterministic heading matcher splits the resume into education / experience / projects / skills / certifications / achievements. Optional `spaCy` NER pulls the candidate's name and organisations; a regex fallback runs if `spaCy` is absent.
- Skill extraction + normalisation** (`ml/skill_extractor.py`, `ml/skills_taxonomy.py`) — a curated taxonomy maps surface forms ("Postgres", "postgresql", "psql") to a single canonical skill ("PostgreSQL") tagged with a category (language / framework / cloud / ...). Extraction is dictionary + phrase matching over the cleaned text, so it is fully explainable: every extracted skill points at the span that produced it.
- JD requirement extraction** (`services/jd_parser.py`) — the JD is split into "required" vs "preferred/nice-to-have" regions using cue phrases ("required", "must have", "preferred", "bonus", "nice to have"), then skills are extracted and normalised the same way. Requirement importance (required > preferred) feeds directly into scoring weights.
- Matching** (`ml/skill_extractor.py` + `ml/similarity.py`) — three signals: *exact* (identical canonical skill), *normalised* (different surface form, same canonical skill), and *semantic* (cosine similarity between embeddings of resume vs JD text/skills, catching "built neural nets in PyTorch" ≈ "deep learning frameworks"). Semantic matches are treated as *related*, never as proof of a skill.
- Scoring** (`scoring/scoring_engine.py`) — a documented weighted sum of six components (see `docs/SCORING.md`). Each component is computed deterministically and returned with its weight and contribution so the UI can show *why*.
- LLM decoration** (`llm/`) — recommendations, interview questions, and a plain-English explanation. Uses the configured provider when a key is set; otherwise deterministic templates built from the actual matched/missing skills. Either way the output is grounded in real pipeline data, which is how we avoid hallucinated advice.

## 5. Key technical decisions & trade-offs

- Deterministic core, LLM leaf.** Testable, explainable, cheap, and works offline. Trade-off: template fallbacks are less fluent than an LLM. Accepted, because correctness and defensibility matter more than prose polish.
- TF-IDF baseline + optional transformer embeddings.** TF-IDF always works (light, no torch) and is a great "did semantics help?" baseline. MiniLM embeddings give real semantic similarity when installed. The `embeddings` module auto-selects and falls back. Trade-off: two code paths, but it keeps the core image small and makes the TF-IDF-vs-embeddings comparison a real, demonstrable thing rather than a slide.
- Curated skill taxonomy vs pure ML skill tagging.** A hand-built taxonomy is explainable and accurate for a known skill universe, and it is honest about its limits (unknown skills are simply not matched). Trade-off: it needs maintenance and won't recognise skills outside the taxonomy — documented as a known limitation.
- Optional persistence.** History uses SQLAlchemy with SQLite by default so the app needs zero database setup to run, and Postgres via `docker-compose` for a production-like setup. Trade-off: a little extra code for a feature that is off the critical path — kept optional behind a flag.

## 6. Data flow of a single request

POST /api/analyze (multipart: resume file + job\_description text) → validate file (type, size) → extract → clean → parse resume → parse JD → extract & normalise skills on both sides → match (exact/normalised/semantic) → score (6 weighted components) →

build strengths/gaps → LLM/template recommendations + interview prep + explanation → assemble `AnalysisResult` → (optional) persist → return JSON. The frontend renders the dashboard from that one JSON object.

## 7. What can go wrong (and how it's handled)

---

Image-only PDF (no text) → friendly 422 with guidance. Empty resume or JD → validated and rejected early. LLM key missing / provider error / timeout → silent fallback to templates, `meta.llm_used=false`. Unknown skills → not matched, surfaced honestly rather than invented. Oversized upload → rejected before parsing. Every failure path returns a structured error, never a raw stack trace.

# Appendix B — Scoring Methodology

---

# Scoring Methodology

The overall match score is **not** an LLM guessing a number. It is a transparent, weighted combination of six components, each computed deterministically and each returned with its own sub-score and explanation. This document is the reference for exactly how the number is produced.

## The formula

$$\text{overall} = 100 \times \sum ( \text{weight}_i \times \text{component}_i ) \qquad \text{where every } \text{component}_i \in [0, 1]$$

| Component                | Weight | What it measures                                                      |
|--------------------------|--------|-----------------------------------------------------------------------|
| Required Skill Coverage  | 0.35   | Fraction of the JD's <b>required</b> skills the resume matches.       |
| Overall Skill Overlap    | 0.20   | Fraction of <b>all</b> named JD skills present on the resume (exact). |
| Preferred Skill Coverage | 0.10   | Fraction of the JD's <b>preferred / nice-to-have</b> skills matched.  |
| Semantic Similarity      | 0.15   | Whole-document meaning overlap (resume vs JD).                        |
| Project Relevance        | 0.10   | How closely the candidate's projects resemble the role.               |
| Experience Relevance     | 0.10   | How closely prior work experience resembles the role.                 |

The weights sum to **1.0** and are enforced at import ( `weights.validate_weights` ). Skill-based signals (required + overlap + preferred = **0.65**) intentionally outweigh the semantic signals (**0.35**): the score stays anchored to concrete skill evidence, while semantics reward relevant experience described in different words than the JD uses.

## How each component is computed

**Required / Preferred coverage.** A skill can be *matched* (exact/normalised — same canonical skill on both sides) or *related* (a curated sibling skill or a high semantic similarity — e.g. resume *PyTorch* for JD *TensorFlow*). A full match earns 1.0; a related-only skill earns partial credit ( `RELATED_CREDIT = 0.5` ). So:

$$\text{required\_coverage} = (\text{required\_matched} + 0.5 \times \text{required\_related}) / \text{required\_total}$$

**Overall skill overlap.** `jd_matched_skills / jd_total_skills`, counting exact matches only — a breadth measure across every skill named in the JD.

**Semantic similarity / Project / Experience relevance.** Cosine similarity from the semantic engine (TF-IDF baseline, or transformer embeddings when installed), then **calibrated** into an interpretable 0-1 range. Raw cosine between two different document types (a resume and a JD) is small even when they're clearly related, and the useful band differs per backend, so each engine rescales its own observed band ( `TfidfEngine` : [0.0, 0.35] → [0, 1]; `EmbeddingEngine` : [0.20, 0.70] → [0, 1] ). Project and experience relevance compare the *projects* and *experience* sections specifically; if a section is missing, the component falls back to a reduced-credit estimate and a warning is attached.

## Why it's explainable

Every component returns {`score`, `weight`, `contribution`, `explanation`}. The dashboard shows the overall number and, directly beneath it, the six contributions that produced it plus a sentence each (e.g. "6/8 required skills matched (+2 related)"). Nothing about the score is a black box: you can always trace an 82 back to which skills were matched, which were only related, and how much semantic context contributed.

## Honest limitations

- Project/experience relevance are semantic by nature and are strongest with the transformer-embedding backend. Under the lightweight TF-IDF fallback they read low when a project describes specifics (e.g. "MNIST digit classifier") that don't lexically overlap the JD — the score stays correctly *ordered*, just compressed. The active backend is reported in the result's `meta`.
- Calibration bands are chosen from observed values, not learned from a labelled dataset — they're a reasonable heuristic, not a trained calibrator.
- A skill outside the taxonomy is invisible to coverage. Coverage therefore measures *recognised* skills, which is why it's paired

with semantic signals.

# Appendix C — Concepts & Design Decisions

---



# Concepts & Design Decisions (Teaching Guide)

---

This document explains every major technical component: what it does, why it's there, how it works, what we considered instead, the trade-offs, what can go wrong, and — for each — **how to explain it in an interview**. Read it top to bottom to understand the whole project.

Only concepts actually used in this project are covered. Where something is optional (embeddings, spaCy, LLM), that is stated.

---

## 1. PDF text extraction

---

**What:** Turn resume PDF bytes into plain text ( `services/pdf_extraction.py` , using `pdfplumber` ).

**Why:** Resumes arrive as PDFs; everything downstream needs text.

**How:** `pdfplumber` reads each page's embedded **text layer** and concatenates it. If the total text is tiny, we assume it's a scanned/image PDF (no text layer) and raise a clear error.

**Why this approach:** Most resumes are exported from Word / Google Docs / LaTeX and contain real text, so direct extraction is fast and accurate — no OCR needed.

**Alternatives:** `PyPDF2` / `pypdf` (also text-layer, sometimes messier spacing); OCR with Tesseract (needed only for scanned PDFs, much heavier and error-prone).

**Trade-offs:** No OCR means scanned resumes aren't supported — a deliberate simplification, surfaced as a friendly error rather than silent garbage.

**What can go wrong:** Encrypted/corrupt PDFs (caught → error), image-only PDFs (detected via near-empty output → error), unusual layouts (columns) can jumble order.

**Interview:** "I use `pdfplumber` to read the PDF's text layer directly, which is accurate for normal exported resumes. I explicitly detect image-only PDFs — where there's no text layer — and return a helpful error instead of feeding garbage into the pipeline. OCR would be the next step if I needed to support scans."

---

## 2. NLP preprocessing (text cleaning)

---

**What:** Normalise raw text before any analysis ( `services/text_cleaning.py` ).

**Why:** PDF text is messy — inconsistent whitespace, words split across lines with hyphens, bullet glyphs, non-breaking spaces. Consistent input makes every later stage more reliable.

**How:** Unify newlines, join hyphenated line-breaks, strip zero-width chars, convert bullets/nbsp to spaces, collapse whitespace. A separate `normalise_for_matching` produces a lowercased form that keeps tokens like `c++` and `node.js` intact for skill matching.

**Alternatives:** Doing nothing (worse matching); heavy NLP normalisation (stemming/lemmatising — unnecessary here and would damage skill tokens).

**Trade-offs:** Rule-based cleaning is simple and predictable but won't fix every exotic PDF artefact.

**Interview:** "Cleaning is a small, deterministic, side-effect-free step that runs once up front. Keeping display text and matching text separate matters — skill matching needs `c++` preserved, but embeddings want natural casing."

---

## 3. Skill extraction & normalisation

---

**What:** Find skills in text and report them under one canonical name ( `ml/skills_taxonomy.py` , `ml/skill_extractor.py` ).

**Why:** "psql", "Postgres", "PostgreSQL" are the same skill. Without normalisation, matching would miss obvious equivalences and coverage would be wrong.

**How:** A curated taxonomy maps alias → canonical + category. Extraction scans text for aliases **longest-first** (so "machine learning" beats "ml"), consuming matched spans, with boundary look-arounds so "ml" can't match inside "html". Short ambiguous tokens ( `c` , `go` , `r` ) only count next to a list delimiter.

**Why this approach:** It's explainable (each match records the surface form and whether normalisation changed it) and accurate for a known skill universe with zero training data.

**Alternatives:** A learned NER/skill tagger (needs labelled data, less predictable, but catches unknown skills); pure fuzzy string matching (noisy).

**Trade-offs:** Needs maintenance and won't recognise skills outside the taxonomy — an honest, documented limitation.

**What can go wrong:** New skills are invisible; ambiguous abbreviations (precision handled by the list-context rule); over-broad aliases (kept conservative).

**Interview:** "I normalise skills through a curated taxonomy — alias to canonical name plus category. Matching is longest-alias-first with word-boundary rules and a special case for short ambiguous tokens like Go or C, which I only accept in a list context. It's fully explainable, which matters more here than catching every obscure skill."

---

## 4. Job-description analysis (required vs preferred)

---

**What:** Split a JD into required vs preferred regions and extract skills from each ( `services/jd_parser.py` ).

**Why:** Not all requirements are equal — missing a *required* skill should hurt the score more than missing a *nice-to-have*.

**How:** Walk the lines, switching a "current bucket" on cue phrases ("required", "must have" vs "preferred", "nice to have", "bonus"). Inline cues like "Required: Python, SQL" keep their content. A skill that's both counts as required.

**Alternatives:** Treating all JD keywords equally (loses signal); an LLM to classify importance (less predictable, unnecessary).

**Trade-offs:** Cue-phrase heuristics can misread unusual JD formats.

**Interview:** "I detect the required vs preferred sections by cue phrases and carry that importance into scoring — required coverage is weighted 0.35, preferred only 0.10."

---

## 5. TF-IDF vs embeddings (the semantic engine)

---

**What:** Represent text as vectors so we can measure similarity ( `ml/embeddings.py` ). Two backends behind one interface.

**TF-IDF** (Term Frequency-Inverse Document Frequency): each document becomes a sparse vector weighting words by how often they appear here vs how rare they are across documents. Similar wording → similar vectors. **Strength:** lightweight, no model download, explainable. **Weakness:** it only sees surface words — "built neural nets in PyTorch" and "deep learning frameworks" share no words, so TF-IDF sees ~0 similarity.

**Embeddings** (sentence-transformers, `all-MiniLM-L6-v2` ): a transformer maps text to a dense vector that captures **meaning**, so those two phrases land close even with no shared words. **Strength:** real semantic similarity. **Weakness:** heavier (pulls in PyTorch), and the model must download once.

**Why both:** TF-IDF always works and is a genuine baseline; embeddings upgrade quality when available. The engine auto-selects ( `SEMANTIC_BACKEND=auto` ) and **calibrates** each backend's raw cosine into a comparable 0-1 range.

**Interview:** "I implemented both TF-IDF and transformer embeddings behind one interface. TF-IDF is a lexical baseline — great when words overlap, blind when they don't. Embeddings capture meaning, so related-but-differently-worded experience is caught. The app auto-selects and falls back, which also let me directly compare the two."

---

## 6. Cosine similarity

---

**What:** The metric comparing two vectors ( `sklearn.metrics.pairwise` and dot products).

**How:** Cosine similarity is the cosine of the angle between two vectors:  $\cos(\theta) = (A \cdot B) / (|A| |B|)$  . It ranges from -1 to 1 (0 to 1 for our non-negative vectors). **It measures direction, not magnitude** — so a short and a long document about the same topic still score as similar, which is exactly what we want (a resume and a JD differ hugely in length).

**Alternatives:** Euclidean distance (sensitive to length/magnitude — bad here); dot product on normalised vectors (equivalent to cosine, which is what the embedding backend uses).

**Interview:** "Cosine similarity compares vector *direction*, ignoring length. That's the right choice for resume vs JD, which differ a

lot in length but can still be about the same thing. I normalise embedding vectors so cosine is just a dot product."

---

## 7. The relatedness ontology

---

**What:** A curated graph of sibling skills ( `skills_taxonomy._RELATED_GROUPS` ) — e.g. {PyTorch, TensorFlow, Keras}, {AWS, GCP, Azure}.

**Why:** For "related" detection to work reliably (and offline), and to give a *human-readable reason* ("both are deep-learning frameworks") instead of just a cosine number.

**How:** If a required JD skill isn't matched, but a resume skill is in its sibling group, it's flagged *related* (partial credit). Embedding similarity is a second, open-ended path to "related".

**Interview:** "I combine a curated skill ontology with embeddings. The ontology gives reliable, explainable relatedness — PyTorch counts as related to a TensorFlow requirement — and embeddings catch the open-ended cases the ontology doesn't list."

---

## 8. Explainable weighted scoring

---

**What:** Combine six components into a 0–100 score, each returned with its sub-score, weight, and reason ( `scoring/` ).

**Why:** The whole point of the project. A defensible score decomposes into named parts tied to evidence.

**How:**  $overall = 100 \times \sum(weight_i \times component_i)$ , weights summing to 1 (enforced at import). Related skills earn partial credit; semantic components are calibrated.

**Alternatives:** Ask an LLM for a number (opaque, unstable, indefensible — the anti-pattern this project deliberately avoids); a trained regressor (needs labelled data).

**Interview:** "The score is a transparent weighted sum, not an LLM guess. Every component has a weight and an explanation, so I can trace an 82 back to which skills matched and how much semantic context contributed. The weights are a documented design choice, validated to sum to 1."

---

## 9. LLM structured output (optional layer)

---

**What:** Use an LLM to generate recommendations, interview questions, and the explanation — as **structured JSON**, grounded in real data ( `llm/` ).

**Why:** LLMs write fluent, tailored prose. But we only let them *phrase* things, never decide the score, and we constrain them to the actual matched/missing skills.

**How:** A provider-agnostic client (OpenAI or Anthropic via `httpx`) sends a system prompt ("only use these skills; return JSON") and parses the JSON. Any failure (no key, timeout, bad JSON) → deterministic template fallback. Every call is time-boxed.

**Avoiding hallucination:** the prompt contains the exact skill lists and instructs the model to use only those; the deterministic fallback is always available; and nothing the LLM returns affects the score.

**Interview:** "The LLM is a leaf, not the trunk. It's optional and swappable between providers, time-boxed, and always falls back to templates. I feed it only the real matched and missing skills and ask for JSON, so it can summarise but not invent, and it never touches the score."

---

## 10. FastAPI (backend framework)

---

**What:** The web framework serving the API ( `app/main.py`, `app/api/` ).

**Why:** Async, fast, Pydantic-based validation, and automatic OpenAPI/Swagger docs — ideal for a typed JSON API. Python keeps the API and the ML pipeline in one language.

**How:** Routes are a thin layer; Pydantic models define the contract; global exception handlers turn errors into clean JSON.

**Alternatives:** Flask (simpler, no built-in validation/async/docs); Django (heavier, more than needed).

**Interview:** "FastAPI gives me typed request/response models, automatic docs, and async out of the box, and keeps the API in the same language as the ML code. I kept the route layer thin so the pipeline is testable without HTTP."

---

## 11. React + TypeScript (frontend)

---

**What:** The single-page UI ( `frontend/` ).

**Why:** Component model fits a dashboard of independent panels; TypeScript gives a typed contract mirroring the API, catching mistakes at compile time.

**How:** A small state machine (form → loading → dashboard), a typed `api.ts` client, and presentational components. The score gauge and bars are pure SVG/CSS — no chart library — for a clean, intentional look and a light bundle.

**Alternatives:** Plain JS (loses type safety); a big component library (heavier, more generic-looking).

**Interview:** "React with TypeScript, and the API response is fully typed so the UI and backend share one contract. I drew the visualisations in SVG rather than pulling in a chart library, which kept the bundle small and the design deliberate."

---

## 12. PostgreSQL / SQLAlchemy (optional persistence)

---

**What:** Optional analysis history ( `app/db/` ).

**Why:** Demonstrates data modelling and lets you list past analyses. Off the critical path — analysis works without it.

**How:** SQLAlchemy ORM, SQLite by default (zero setup), PostgreSQL via docker-compose. Only the derived result is stored — no raw resume text — keeping personal data out of the DB. Saving is best-effort and never breaks a request.

**Alternatives:** No persistence (loses the feature); a full ORM migration setup (overkill here).

**Interview:** "History is optional and uses SQLAlchemy — SQLite locally, Postgres in Docker. I store only the derived result, not the resume, so I'm not persisting personal data unnecessarily, and a DB failure degrades gracefully."

---

## 13. Docker & docker-compose

---

**What:** Containerisation and one-command full-stack startup.

**Why:** Reproducible environments; "works on my machine" solved; easy deploy.

**How:** A slim Python image for the backend (multi-arg to optionally bake in the ML stack), a multi-stage Node→nginx image for the frontend, and a compose file wiring backend + frontend + Postgres, with the frontend's nginx proxying `/api`.

**Interview:** "The backend image is intentionally light by default with an opt-in build arg for the heavy ML stack — a real size-vs-capability trade-off. The frontend is a multi-stage build: Node compiles it, nginx serves the static output and proxies the API."

---

## 14. Security practices

---

**What:** Basic production hygiene.

**How:** Secrets via env vars, never committed ( `.env` git-ignored, `.env.example` provided); uploads validated (extension, declared type, size, and the real `%PDF` magic number) before any parsing; uploaded files are never executed; only derived results are stored; CORS restricted to configured origins; the container runs as a non-root user.

**Interview:** "I validate uploads by extension, size, and the actual PDF magic bytes before touching them, keep secrets in the environment, restrict CORS, run the container as non-root, and never persist the raw resume — only the analysis result."

---

## 15. Testing

---

**What:** 38 pytest tests ( `backend/tests/` ).

**How:** Unit tests for cleaning, extraction/normalisation, JD parsing, matching, and scoring (including edge cases — empty input, no

required skills, boundary false positives), plus API tests via FastAPI's `TestClient` (happy paths and error paths). Tests force the TF-IDF backend and disable the LLM so they're fast, deterministic, and network-free.

**Interview:** "I test the pipeline layer by layer without needing a server, plus API tests through the `TestClient`. The interesting ones are the edge cases — that 'ml' doesn't match inside 'html', that a bad PDF returns a clean error, that a better-fitting resume always scores higher."

---

## 16. Deployment

---

Covered in `DEPLOYMENT.md`: backend as a container, frontend as static hosting, optional managed Postgres, and a pre-deploy checklist (health check, CORS origins, `VITE_API_BASE` set at build time, secrets not committed).

**Interview:** "Backend as a container on something like Render, frontend as a static build on Vercel with the API base baked in at build time, and I make sure the backend's CORS list includes the exact frontend origin."